

FE Evening Exam — Pseudocode Collection (Last 4 Years)

A comprehensive study guide covering all pseudocode questions from the **April 2024, October 2024, April 2025, and October 2025** FE evening exams. Questions are organized by difficulty level and grouped into logical categories so you can build your skills progressively.

How to Use This Guide

- **Level 1 — Fundamentals:** Simple logic, basic arithmetic, and straightforward control flow. If you're just starting, begin here.
- **Level 2 — Intermediate:** Involves loops with more nuance, basic data structures (arrays, stacks), string operations, and well-known algorithms like sorting and searching.
- **Level 3 — Advanced:** Complex data structures (trees, linked lists, graphs, hash tables), recursion, mathematical series, and multi-step algorithms.

Each question includes:

-  **Year & Exam** — which exam it appeared in
-  **Explanation** — a plain-language walkthrough with analogies and examples
-  **Pseudocode** — the exact exam pseudocode

Level 1 — Fundamentals

Simple, easy-to-follow code. These problems test basic conditional logic, simple loops, and arithmetic operations. Great for building your confidence before moving to harder problems.

Category: Conditional Logic & Simple Decisions

1.1 — Letter Grade Evaluation

 Year & Exam: April 2024 (2024S)

 Explanation:

Think of this like a ticket booth at a theme park — depending on your height (score), you get a different wristband (grade):

- Score **80–100** → Grade **"D"** (Distinction)
- Score **50–79** → Grade **"P"** (Pass)
- Score **0–49** → Grade **"F"** (Fail)

The function checks from the highest threshold down. Once a condition matches, it assigns the grade and stops. This is a classic **if-elseif-else chain** — a pattern you'll see everywhere in programming.

Example: `grade(85)` → `"D"`, `grade(55)` → `"P"`, `grade(30)` → `"F"`

 Pseudocode:

```

o character: grade (integer: score)
character: ret
if (score ≥ 80)
    ret ← "D"
elseif (score ≥ 50)
    ret ← "P"
else
    ret ← "F"
endif
return ret

```

1.2 — Bus Ticket Price Calculation

 Year & Exam: April 2024 (2024S)

 Explanation:

Imagine you're buying a bus ticket. The standard fare is , but you get a **50% discount** (`20 • • , butyougeta • • 5010`) if any of these apply:

- You're **10 years old or younger** (child discount)
- You're **60 years old or older** (senior discount)
- You're a **registered member** (loyalty discount)

The key insight is the use of the **OR** operator — if *any one* of these conditions is true, you get the discount. This is a common real-world pattern: multiple conditions leading to the same outcome.

Example: `ticketPrice(8, false)` → `10`, `ticketPrice(35, true)` → `10`, `ticketPrice(35, false)` → `20`

 Pseudocode:

```

o integer: ticketPrice(integer: age, boolean: isMember)
  integer: ret
  if ((age ≤ 10) or (age ≥ 60)) or isMember
    ret ← 10
  else
    ret ← 20
  endif
  return ret

```

1.3 — Leap Year Verification

 **Year & Exam:** April 2025 (2025S)

 **Explanation:**

Leap years aren't as simple as "divisible by 4." There's a hierarchy of rules — think of it like a **decision tree with exceptions**:

1. **Divisible by 400?** →  Leap year (e.g., 2000)
2. **Divisible by 100?** →  NOT a leap year (e.g., 1900)
3. **Divisible by 4?** →  Leap year (e.g., 2024)
4. **Otherwise** →  NOT a leap year (e.g., 2023)

The order matters! The function checks the most specific rule first (400), then the exception (100), then the general rule (4). This is a great example of **order-dependent conditional logic**.

Example: `isLeapYear(2000)` → `true`, `isLeapYear(1900)` → `false`, `isLeapYear(2024)` → `true`

 **Pseudocode:**

```

o boolean: isLeapYear (integer: year)
  if (year mod 400 = 0)
    return true
  elseif (year mod 100 = 0)
    return false
  elseif (year mod 4 = 0)
    return true
  else
    return false
  endif

```

1.4 — Coupon Calculation

 **Year & Exam:** October 2024 (2024A)

 **Explanation:**

Think of a grocery store promotion: "Buy 3 of product X, get 1 coupon — but only if the product ID ends in 3."

The function uses two simple operations:

- `prod_id mod 10` — extracts the last digit of the product ID (e.g., 123 → 3). This is like looking at only the ones place.
- `pur_prod / 3` (integer division) — counts how many groups of 3 were purchased (e.g., buying 7 gives 2 coupons, with 1 leftover item).

If the last digit isn't 3, you get zero coupons regardless.

Example: `coupon(123, 7)` → `2`, `coupon(125, 10)` → `0`

 **Pseudocode:**

```

o integer: coupon (integer: prod_id, integer: pur_prod)
  integer: num_coupon ← 0
  if (prod_id mod 10 = 3)
    num_coupon ← integer part of (pur_prod / 3)
  endif
  return num_coupon

```

Category: Simple Arithmetic & Math

1.5 — Calculation of $\sqrt{x + y}$

```
c-2.7,0,-7.17,-2.7,-13.5,-8c-5.8,-5.3,-9.5,-10,-9.5,-14
c0,-2,0.3,-3.3,1,-4c1.3,-2.7,23.83,-20.7,67.5,-54
c44.2,-33.3,65.8,-50.3,66.5,-51c1.3,-1.3,3,-2.5,-2c4.7,0,8.7,3.3,12,10
s173,378,173,378c0.7,0,35.3,-71,104,-213c68.7,-142,137.5,-285,206.5,-429
c69,-144,104.5,-217.7,106.5,-221 10 -0 c5.3,-9.3,12,-14,20,-14 H400000v40H845.2724 s-225.272,467,-225.272,467s-235,486,-235,486c-
2.7,4.7,-9.7,-19,7 c-6,0,-10,-1,-12,-3s-194,-422,-194,-422s-65,47,-65,47z M834 80h400000v40h-400000z"/> + y
√
)2
```

 **Year & Exam:** April 2025 (2025S)

 **Explanation:**

This is a one-liner that uses a **power function** to compute square roots. The key math trick:

- $\sqrt{x} = x^{0.5}$ — raising to the power of 0.5 is the same as taking the square root.
- So `pow(x, 0.5)` gives $\sqrt{x}$, and `pow(y, 0.5)` gives $\sqrt{y}$.
- Then we add them and square the result: `pow(sum, 2)`.

It's mathematically equivalent to $x + y + 2\sqrt{xy}$, but the pseudocode computes it directly.

Example: `calc(4, 9)` → `pow(2 + 3, 2)` → `pow(5, 2)` → `25`

 **Pseudocode:**

```
◦ real: calc(real: x, real: y)
  return pow(pow(x, 0.5) + pow(y, 0.5), 2)
```

1.6 — Sum of Even Numbers (1 to 100)

 **Year & Exam:** April 2024 (2024S)

 **Explanation:**

This program walks through every number from 1 to 100 and asks: "Is this number even?" The **modulus operator** (`mod 2`) is the key tool — if the remainder when dividing by 2 is 0, the number is even.

Think of it like a filter: you scan every item on a conveyor belt, and only pick up the even ones. After collecting all even numbers (2, 4, 6, ..., 100), you add them all up.

The expected output is all even numbers followed by their sum: **2550**.

 **Pseudocode:**

```
integer: i
integer: sum ← 0
for (increase i from 1 to 100 by 1)
  if (i mod 2 = 0)
    output i
    sum ← sum + i
  endif
endfor
output sum
```

1.7 — Sum of Digits

 **Year & Exam:** April 2024 (2024S)

 **Explanation:**

Imagine you have the number **4821** and want to add up all its digits: $4 + 8 + 2 + 1 = 15$. This function does exactly that using a "peel off the last digit" technique:

1. `num mod 10` — grabs the last digit (4821 → 1)

2. `num / 10` (integer division) — chops off the last digit (4821 → 482)
3. Repeat until there are no digits left (`num` becomes 0)

Think of it like reading digits off a number from right to left, adding each one to a running total. This digit-extraction pattern appears frequently in programming.

Example: `sumDigits(4821)` → 4 + 8 + 2 + 1 = `15`

 **Pseudocode:**

```

integer: sumDigits (integer: num)
integer: sum ← 0
while (num > 0)
    sum ← sum + num mod 10
    num ← integer part of (num / 10)
endwhile
return sum

```

1.8 — Seconds to HH:MM:SS Conversion

 **Year & Exam:** April 2024 (2024S)

 **Explanation:**

Converting raw seconds into hours, minutes, and seconds is like making change with coins of different denominations:

- **Hours** = total seconds ÷ 3600 (how many full hours fit?)
- **Minutes** = (total seconds ÷ 60) mod 60 (how many full minutes remain after removing hours?)
- **Seconds** = total seconds mod 60 (what's left after removing full minutes?)

For **5450 seconds**: 5450 ÷ 3600 = 1 hour, (5450 ÷ 60) mod 60 = 90 mod 60 = 30 minutes, 5450 mod 60 = 50 seconds → Output: `1, 30, 50`

The helper functions `division` and `modulus` simply wrap the basic integer division and modulus operations.

 **Pseudocode:**

```

convert(integer: input)
integer: hour, minute, second
second ← modulus (input, 60)
minute ← modulus (division (input, 60), 60)
hour ← division (input, 3600)
output hour, minute, second

integer: division (integer: a, integer: b)
integer: u
u ← integer part of (a / b)
return u

integer: modulus (integer: a, integer: b)
integer: u
u ← a mod b
return u

```

1.9 — Sum of Even Elements in Range

 **Year & Exam:** October 2025 (2025A)

 **Explanation:**

This is a combination of two simple ideas:

1. **Range filtering** — only look at array elements from index `k` to index `m`
2. **Even check** — only add up the elements that are even (`mod 2 = 0`)

Think of it like walking down a shelf in a library: you start at position `k`, end at position `m`, and only pick up books with even page counts.

Example: Given array `{3, 8, 5, 12, 7, 6}`, calling `sum(array, 2, 5)` checks elements at indices 2–5: 8 (even ✓), 5 (odd ✗), 12 (even ✓), 7 (odd ✗) → returns `20`.

 **Pseudocode:**

```

o integer: sum(integer []: array, integer: k, integer: m)
integer: s ← 0
integer: i ← k
while (i ≤ m)
  if (array[i] mod 2 = 0)
    s ← s + array[i]
  endif
  i ← i + 1
endwhile
return s

```

1.10 — Reverse Digits of a Number

 **Year & Exam:** October 2025 (2025A)

 **Explanation:**

This uses the same "peel off the last digit" technique as the Sum of Digits problem, but instead of adding digits, we **build a new number in reverse**:

1. **Extract** the last digit: `temp mod 10`
2. **Append** it to the reversed number: `rev * 10 + rm` (shifting existing digits left and adding the new one)
3. **Chop** the last digit off: `temp / 10`

Think of it like reading a phone number backwards and writing each digit down. Each time you write a new digit, you shift the previous ones to the left.

Example: 456 → extract 6, rev = 6 → extract 5, rev = 65 → extract 4, rev = 654

 **Pseudocode:**

```

o reverse_digits (integer: num)
integer: rev ← 0
integer: rm ← 0
integer: temp ← num
while (temp > 0)
  rm ← temp mod 10
  rev ← rev * 10 + rm
  temp ← integer part of (temp / 10)
endwhile
output "Number is", num, ", output", rev

```

1.11 — Perfect Number Verification

 **Year & Exam:** April 2025 (2025S)

 **Explanation:**

A **perfect number** is a number that equals the sum of all its proper divisors (divisors excluding itself). It's like a number that's "perfectly balanced."

The function checks every number from 1 up to `n/2` (no divisor other than `n` itself can be larger than `n/2`) and tests if it divides evenly into `n`. If so, it adds it to a running sum. At the end, if the sum equals `n`, the number is perfect.

Famous perfect numbers: 6 (1+2+3 = 6), 28 (1+2+4+7+14 = 28), 496, 8128

 **Pseudocode:**

```

◦ boolean: isPerfect (integer: n)
  integer: k
  integer: sum ← 0
  integer: half ← integer part of (n / 2)
  for (increase k from 1 to half by 1)
    if (n mod k = 0)
      sum ← sum + k
    endif
  endfor
  if (sum = n)
    return true
  else
    return false
  endif

```

1.12 — Geometric Mean

 **Year & Exam:** October 2024 (2024A)

 **Explanation:**

While the **arithmetic mean** (regular average) adds numbers and divides, the **geometric mean** multiplies numbers and takes the n-th root. It's commonly used for growth rates and ratios.

For numbers $a_1, a_2, \dots, a_n$:

$$\text{Geometric Mean} = \sqrt[n]{a_1 \times a_2 \times \dots \times a_n} = (a_1 \times a_2 \times \dots \times a_n)^{1/n}$$

The function multiplies all elements together, then raises the product to the power of $1/n$.

Example: For `{2, 8}`: product = 16, geometric mean = $16^{1/2} = 4$

 **Pseudocode:**

```

◦ real: calcGeoMean (real []: dataArray)
  real: product, geomean
  integer: n ← the number of elements in dataArray
  integer: i
  product ← 1
  for (increase i from 1 to n by 1)
    product ← product * dataArray[i]
  endfor
  geomean ← pow(product, 1 / n)
  return geomean

```

1.13 — Quadratic Equation Roots

 **Year & Exam:** October 2024 (2024A)

 **Explanation:**

The classic quadratic formula you learned in math class, implemented as code! For the equation $ax^2 + bx + c = 0$:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The **discriminant** ($b^2 - 4ac$) tells us how many solutions exist:

- **Positive discriminant** → two distinct roots (the parabola crosses the x-axis twice)
- **Zero discriminant** → one repeated root (the parabola just touches the x-axis)

Think of the discriminant as a "traffic light": positive = two paths, zero = one path.

Example: For $x^2 - 5x + 6 = 0$ (a=1, b=-5, c=6): discriminant = $25 - 24 = 1$, roots = 3 and 2

 **Pseudocode:**

```
◦ findRoots (real: a, real: b, real: c)
  real: discriminant, root1, root2
  discriminant ← b * b - 4 * a * c
  if (discriminant > 0)
    root1 ← (-b + sqrt(discriminant)) / (2 * a)
    root2 ← (-b - sqrt(discriminant)) / (2 * a)
    output "root1 = ", root1, " and root2 = ", root2
  elseif (discriminant = 0)
    root1 ← -b / (2 * a)
    output "root1 = root2 = ", root1
  endif
```

Level 2 — Intermediate

Slightly harder code involving well-known algorithms, array manipulation, basic data structures (stacks), string processing, and number theory. These require understanding loops, nested logic, and algorithmic patterns.

Category: Number Theory & Bit Manipulation

2.1 — Binary to Decimal Conversion

 **Year & Exam:** April 2025 (2025S)

 **Explanation:**

This function treats a decimal number (like 1100) as if its digits represent a binary number, and converts it to the equivalent decimal value.

Think of it like reading a binary number digit by digit from right to left, where each position has a **weight** that doubles (1, 2, 4, 8, 16...):

STEP	DIGIT EXTRACTED	WEIGHT	CONTRIBUTION
1	0 (from 1100)	1	0
2	0 (from 110)	2	0
3	1 (from 11)	4	4
4	1 (from 1)	8	8

Total: $0 + 0 + 4 + 8 = 12$

The `mod 10` extracts the rightmost digit, and `/ 10` shifts the number right.

 **Pseudocode:**

```

integer: convert(integer: number)
integer: place, n, remainder, decimal
decimal ← 0
place ← 1
n ← number
while (n > 0)
    remainder ← n mod 10
    n ← integer part of (n / 10)
    decimal ← decimal + remainder * place
    place ← 2 * place
endwhile
return decimal

```

2.2 — Print Prime Numbers

 **Year & Exam:** October 2025 (2025A)

 **Explanation:**

A **prime number** is only divisible by 1 and itself. This procedure finds the first N primes using the **trial division** method.

The clever optimization: you only need to check divisors up to the **square root** of the number. Why? If a number `n` has a divisor larger than $\sqrt{n}$, then it must also have a corresponding divisor smaller than $\sqrt{n}$. So checking up to $\sqrt{n}$ is sufficient.

Think of it like checking if a number has a "partner" — if no small partner exists, no large one does either.

Example: `printPrime(6)` → checks 2 , 3 , 4  (2×2), 5 , 6  (2×3), 7 , ..., 11 , ..., 13  → prints `"2 3 5 7 11 13"`

 **Pseudocode:**

```

◦ printPrime (integer: N)
  integer: count, number, i
  boolean: isPrime
  count ← 1
  number ← 2
  while (count ≤ N)
    isPrime ← true
    for (increase i from 2 to integer part of the square root of number by 1)
      if (number mod i = 0)
        isPrime ← false
        exit the for block
      endif
    endfor
    if (isPrime = true)
      output number, " "
      count ← count + 1
    endif
    number ← number + 1
  endwhile

```

2.3 — Prime Factors

 **Year & Exam:** October 2024 (2024A)

 **Explanation:**

Every integer greater than 1 can be broken down into a product of prime numbers — this is called **prime factorization**. The algorithm works like peeling layers off an onion:

1. Start with the smallest prime (2)
2. If it divides the number evenly, record it and divide the number
3. If not, move to the next candidate
4. Repeat until the number is fully broken down (becomes 1)

The beauty of this approach is that you don't need a separate "is this prime?" check. Non-primes (like 4, 6, 9) will never appear as factors because their prime components are extracted first.

Example: `primeFactors(78)` : $78 \div 2 = 39$, $39 \div 3 = 13$, $13 \div 13 = 1 \rightarrow$ Output: `"2 x 3 x 13"`

 **Pseudocode:**

```

◦ primeFactors (integer: num)
  integer: i
  i ← 2
  do
    if (num mod i = 0)
      num ← integer part of (num / i)
      output i
      if (num > 1)
        output "x"
      endif
    else
      i ← i + 1
    endif
  while (num > 1)

```

2.4 — Count Set Bits

 **Year & Exam:** April 2024 (2024S)

 **Explanation:**

This function counts how many bits are "1" in an 8-bit binary number. Think of it like counting how many lights are ON in a row of 8 light switches.

It uses **bitwise operations**:

- `00000001 << (i-1)` creates a "mask" — a number with only the i-th bit set to 1 (like pointing a flashlight at one specific switch)
- `rbyte & mask` performs a bitwise AND — if the result isn't zero, that bit is ON

ITERATION	MASK	PURPOSE
i=1	00000001	Check bit 1 (rightmost)
i=2	00000010	Check bit 2
i=3	00000100	Check bit 3
...	...	...
i=8	10000000	Check bit 8 (leftmost)

Example: `count1(11001011)` → bits 1, 2, 4, 7, 8 are ON → returns `5`

 Pseudocode:

```

integer: count1(bit8: byte)
bit8: rbyte ← byte
integer: r ← 0
integer: i
for (increase i from 1 to 8 by 1)
  if (rbyte & (00000001 << (i - 1)) ≠ 00000000)
    r ← r + 1
  endif
endfor
return r

```

2.5 — Gray Code to Binary Conversion

 Year & Exam: April 2025 (2025S)

 Explanation:

Gray code is a special binary encoding where consecutive values differ by only 1 bit (useful in reducing errors in physical systems like rotary encoders).

The conversion algorithm works by repeatedly XOR-ing the value with a right-shifted version of itself:

- `z >> 1` shifts all bits one position to the right (like dividing by 2 in binary)
- `y ^ z` performs XOR — the "different detector" that outputs 1 only when bits differ

The loop continues shifting and XOR-ing until the shifted value becomes all zeros. Each iteration "corrects" one more bit position.

Example: Gray code `00001100` → Binary `00001000`

 Pseudocode:

```

8-bit: GrayBiCon(8-bit: x)
8-bit: y ← x
8-bit: z ← x
while (z ≠ 00000000)
  z ← z >> 1
  y ← y ^ z
endwhile
return y

```

2.6 — Fibonacci Sequence

 Year & Exam: April 2024 (2024S)

 Explanation:

The Fibonacci sequence is nature's favorite number pattern — each number is the sum of the two before it: **0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...**

This implementation uses **recursion** — the function calls itself, like looking up a word in a dictionary only to find it defined using another word you also need to look up.

- **Base cases:** Position 1 → returns 0, Position 2 → returns 1
- **Recursive case:** `fibonacci(n) = fibonacci(n-1) + fibonacci(n-2)`

Think of it like asking "what's the 9th Fibonacci number?" which becomes "what's the 8th + the 7th?" and each of those ask further questions, all the way down to the base cases.

Example: `fibonacci(9)` → `fibonacci(8) + fibonacci(7)` → ... → `21`

 **Pseudocode:**

```
integer: fibo (integer: n)
  if ((n = 1) or (n = 2))
    return n - 1
  else
    return fibo(n - 1) + fibo(n - 2)
  endif
```

Category: String & Array Operations

2.7 — Palindrome Check (String)

 **Year & Exam:** April 2024 (2024S)

 **Explanation:**

A **palindrome** reads the same forwards and backwards (like "MADAM" or "RACECAR"). The algorithm uses the **two-pointer technique**:

1. Place one pointer at the **start** (i) and one at the **end** (j)
2. Compare the characters at both pointers
3. If they match, move both pointers inward
4. If they don't match, it's NOT a palindrome — stop immediately

This is efficient because it only checks half the string (the minimum number of comparisons needed). Think of it like two people reading the same word — one from the left, one from the right — checking if they always agree.

Example: "MADAM" → M=M ✓, A=A ✓, middle D reached → palindrome!

 **Pseudocode:**

```
isPalindrome (string: str)
  integer: i, j, len
  boolean: flag
  flag ← true
  len ← number of characters in str
  i ← 1
  j ← len
  while (i < j)
    if (the i-th character of string str ≠ the j-th character of string str)
      flag ← false
      exit the while block
    endif
    i ← i + 1
    j ← j - 1
  endwhile
  if (flag)
    output str, " is a palindrome."
  else
    output str, " is not a palindrome."
  endif
```

2.8 — Palindrome Check (Array)

 **Year & Exam:** October 2024 (2024A)

 **Explanation:**

Same concept as the string palindrome check above, but operates on a **character array** instead of a string. The two-pointer technique is identical:

- `left` starts at the beginning, `right` starts at the end
- If characters match, move inward; if not, return `false`

This version returns a boolean (`true` / `false`) rather than printing a message, making it more reusable as a utility function.

Example: `{'n','o','o','n'}` → `n=n` ✓, `o=o` ✓ → returns `true`

 Pseudocode:

```

o boolean: isPalindrome (character []: s)
  integer: left ← 1
  integer: right ← the number of elements in s
  boolean: ok ← true
  while (left < right)
    if (s[left] = s[right])
      left ← left + 1
      right ← right - 1
    else
      ok ← false
      break
    endif
  endwhile
  return ok

```

2.9 — Hamming Distance

 Year & Exam: April 2025 (2025S)

 Explanation:

Hamming distance measures how "different" two sequences are by counting the positions where they disagree. It's widely used in error detection (e.g., detecting transmission errors in data).

Think of it like comparing two exam answer sheets side by side. You go position by position, and every time the answers differ, you add 1 to your count.

Important constraint: Both arrays must be the same length. If they aren't, the function returns -1 as an error signal.

Example: `hammingDistance({'A','B','C'}, {'A','X','C'})` → position 2 differs → returns `1`

 Pseudocode:

```

o integer: hammingDistance (character []: s1, character []: s2)
  integer: i, cnt ← 0
  if (the number of elements in s1 ≠ the number of elements in s2)
    return -1
  endif
  for (increase i from 1 to the number of elements in s1 by 1)
    if (s1[i] ≠ s2[i])
      cnt ← cnt + 1
    endif
  endfor
  return cnt

```

2.10 — Contains Substring

 Year & Exam: October 2025 (2025A)

 Explanation:

This function checks whether one character sequence appears inside another — like using Ctrl+F to find a word in a document. It implements the **naive string matching algorithm**:

1. Slide a "window" of size `seqLen` across the main string
2. At each position, compare characters one by one
3. If all characters in the window match the sequence → found it!
4. If any character doesn't match → slide the window forward and try again

The variable `len = strlen - seqLen + 1` limits how far the window can slide (no point starting a comparison if there aren't enough characters left).

Example: `containsSubstring("HELLO WORLD", "LO W")` → found starting at position 4 → returns `true`

 Pseudocode:

```

◦ boolean: containsSubstring(character []: str, character []: seq)
integer: strlen, seqlen, len, i, j
strlen ← the number of elements in str
seqlen ← the number of elements in seq
if (seqlen > strlen)
    return false
endif
len ← strlen - seqlen + 1
for (increase i from 1 to len by 1)
    j ← 1
    while (j ≤ seqlen)
        if (str[i + j - 1] ≠ seq[j])
            exit the while block
        endif
        j ← j + 1
    endwhile
    if (j = seqlen + 1)
        return true
    endif
endfor
return false

```

2.11 — Find Mode of Dataset

 **Year & Exam:** October 2025 (2025A)

 **Explanation:**

The **mode** is the most frequently occurring value in a dataset — like finding the most popular flavor at an ice cream shop by tallying orders.

The algorithm uses a **brute-force counting approach**:

1. For each element, count how many times it appears in the rest of the array
2. Keep track of the element with the highest count so far
3. The winner at the end is the mode

This uses a nested loop — the outer loop picks a candidate, and the inner loop counts its occurrences.

Example: `findMode({2, 1, 1, 9, 6, 6, 2, 5, 6})` → 6 appears 3 times (most) → returns `6`

 **Pseudocode:**

```

◦ integer findMode (integer []: arr)
integer: n ← the number of elements in arr
integer: m ← arr[1] /* Current mode value */
integer: m_c ← 1 /* Frequency count of mode */
integer: c, i, j
for (increase i from 1 to n - 1 by 1)
    c ← 1
    for (increase j from i + 1 to n by 1)
        if (arr[i] = arr[j])
            c ← c + 1
        endif
    endfor
    if (m_c < c)
        m_c ← c
        m ← arr[i]
    endif
endfor
return m

```

2.12 — Second-Largest Element

 **Year & Exam:** October 2024 (2024A)

 **Explanation:**

Finding the second-largest element with a **single pass** through the array. The trick is to maintain two trackers simultaneously:

- `max1` — the largest value seen so far

- `max2` — the second-largest value seen so far

When you find a new maximum, the old maximum "demotes" to become the second maximum. If a value isn't the largest but beats the second-largest, only `max2` updates.

Think of it like a podium at a race: when a new champion appears, the old champion slides to silver.

Example: `{2, 6, 9, 1, 7, 5}` → `max1=9, max2=7` → output `7`

 Pseudocode:

```
integer []: array ← {2, 6, 9, 1, 7, 5}
integer: i
integer: max1 ← -∞
integer: max2 ← -∞
for (increase i from 1 to the number of elements of array by 1)
  if (array[i] > max1)
    max2 ← max1
    max1 ← array[i]
  elseif (array[i] > max2)
    max2 ← array[i]
  endif
endfor
output max2
```

Category: Sorting Algorithms

2.13 — Selection Sort

 Year & Exam: April 2024 (2024S)

 Explanation:

Selection sort works like organizing a hand of playing cards: repeatedly find the smallest card and place it at the leftmost unsorted position.

1. Scan the entire array to find the **minimum**
2. Swap it with the first element
3. Now the first position is "sorted" — repeat for the remaining positions

It's simple and intuitive but slow for large arrays ($O(n^2)$) because it scans the remaining elements every time.

Example: `{12, 11, 13, 5, 6}` → find min 5, swap with 12 → `{5, 11, 13, 12, 6}` → find min 6, swap with 11 → `{5, 6, 13, 12, 11}` → ...

 Pseudocode:

```
integer []: data ← {12, 11, 13, 5, 6}
integer: i, j, temp, minPos
integer: size ← the number of elements in data
for (increase i from 1 to (size - 1) by 1)
  minPos ← i
  for (increase j from i + 1 to size by 1)
    if (data[j] < data[minPos])
      minPos ← j
    endif
  endfor
  temp ← data[i]
  data[i] ← data[minPos]
  data[minPos] ← temp
endfor
```

2.14 — Insertion Sort

 Year & Exam: October 2024 (2024A)

 Explanation:

Insertion sort works like sorting a hand of cards as you pick them up: each new card is inserted into its correct position among the already-sorted cards.

For each element starting from the second:

1. Compare it with the element to its left
2. If it's smaller, **swap** them and move one step left
3. Keep swapping until it's in the right spot (or reaches the beginning)

It's efficient for **nearly sorted** data because few swaps are needed.

Example: {5, 3, 8, 1} → insert 3: {3, 5, 8, 1} → 8 stays → insert 1: {1, 3, 5, 8}

 **Pseudocode:**

```

◦ sort (integer []: arg)
  integer []: A ← arg
  integer: i, k
  for (increase i from 2 to the number of elements in A by 1)
    k ← i
    while (k > 1)
      if (A[k - 1] ≤ A[k])
        exit the while block
      endif
      swap A[k] and A[k - 1]
      k ← k - 1
    endwhile
  endfor
  output A

```

2.15 — Optimized Bubble Sort

 **Year & Exam:** October 2025 (2025A)

 **Explanation:**

Standard **bubble sort** repeatedly walks through the array, comparing adjacent pairs and swapping them if they're out of order. Like bubbles rising to the surface, the largest unsorted element "floats" to the top each pass.

This **optimized version** adds an **exchange** flag:

- If no swaps occur during a full pass, the array is already sorted → **stop early!**
- This avoids unnecessary passes over already-sorted data

After each pass, the last element is guaranteed to be in its correct position, so **maxIdx** decreases by 1.

Example: {3, 1, 4, 1} → Pass 1: swap 3 ↔ 1 → {1, 3, 4, 1}, swap 4 ↔ 1 → {1, 3, 1, 4} → Pass 2: swap 3 ↔ 1 → {1, 1, 3, 4} → Pass 3: no swaps → done!

 **Pseudocode:**

```

◦ integer []: quickBubble (integer []: array)
  integer []: arraySorted ← array
  boolean: exchange
  integer: maxIdx, i
  exchange ← true
  maxIdx ← (the number of elements in arraySorted) - 1
  while (maxIdx > 0 and exchange = true)
    exchange ← false
    for (increase i from 1 to maxIdx by 1)
      if (arraySorted[i] > arraySorted[i+1])
        exchange ← true
        swap the values of arraySorted[i] and arraySorted[i+1]
      endif
    endfor
    maxIdx ← maxIdx - 1
  endwhile
  return arraySorted

```

2.16 — Counting Sort

 **Year & Exam:** April 2025 (2025S)

 Explanation:

Counting sort is a non-comparison sort — instead of comparing elements to each other, it **counts how many times each value appears**, then reconstructs the sorted output from the counts.

Think of it like sorting mail into numbered mailboxes:

1. Walk through the unsorted pile — drop each letter into its numbered mailbox
2. Then walk through the mailboxes in order (0, 1, 2, ...) and collect the mail

Key constraint: This only works when values fall within a known, small range (here: 0–10).

Example: {9, 3, 2, 0, 9, 3, 0, 1, 5, 3, 8} → counts: s[0]=2, s[1]=1, s[2]=1, s[3]=3, s[5]=1, s[8]=1, s[9]=2 → output: "0, 0, 1, 2, 3, 3, 3, 5, 8, 9, 9, "

 Pseudocode:

```

◦ sort(integer []: arr) // prints all the elements in arr in ascending order
integer []: s ← {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0}
integer: i, j
for (increase i from 1 to (the number of elements in arr) by 1)
    s[arr[i]] ← s[arr[i]] + 1
endfor
for (increase i from 0 to 10 by 1)
    if (s[i] > 0)
        for (increase j from 0 to s[i] - 1 by 1)
            output i and ", "
        endfor
    endif
endfor
return

```

Category: Stack Operations

2.17 — Stack Push and Pop Operations

 **Year & Exam:** October 2025 (2025A)

 Explanation:

A **stack** is a Last-In-First-Out (LIFO) data structure — like a stack of plates. You can only add to the top (**push**) or remove from the top (**pop**).

This implementation uses:

- An array **stck** of size 10 to hold elements
- A pointer **tos** (top of stack) that tracks where the top element is

Key behaviors:

- **push** : If the stack isn't full (**tos < 10**), increment **tos** and store the item
- **pop** : If the stack isn't empty (**tos ≥ 1**), return the top item and decrement **tos**

Think of **tos** as a bookmark that tells you which plate is on top.

 Pseudocode:

```

global: integer []: stck ← {10 undefined}
global: integer: tos ← 0

◦ push (integer: item)
  if (tos = 10)
    output "Stack is Full"
  else
    tos ← tos + 1
    stck[tos] ← item
  endif

◦ integer: pop()
  integer: item
  if (tos < 1)
    output "Stack is Empty"
    return undefined
  else
    item ← stck[tos]
    tos ← tos - 1
  endif
  return item

```

2.18 — Stack Operations (with Full/Empty checks)

 **Year & Exam:** April 2024 (2024S)

 **Explanation:**

This is another stack implementation, but with explicit `empty()` and `full()` helper functions. The stack uses an array of size 4 and `index` starts at 1 (the next available slot).

Key difference from 2.17: Here, `index` points to the **next empty slot** (not the top element). So:

- `push` : stores the item at `content[index]`, then increments `index`
- `pop` : decrements `index` first, then returns `content[index]`

Returns `true` on successful push, `false` if full. Returns `-1` from pop if empty.

 **Pseudocode:**

```

global: integer []: content ← {undefined, undefined, undefined, undefined}
global: integer: index ← 1
global: integer: max ← 4 /* max size of the stack */

◦ boolean: empty()
  if (index = 1)
    return true
  else
    return false
  endif

◦ boolean: full()
  if (index > max)
    return true
  else
    return false
  endif

◦ boolean: push (integer: i)
  if (not full())
    content[index] ← i
    index ← index + 1
    return true
  else
    return false
  endif

◦ integer: pop()
  if (not empty())
    index ← index - 1
    return content[index]
  else
    return -1
  endif

```

2.19 — String Reverse Using Stack

 Year & Exam: April 2025 (2025S)

 Explanation:

This uses a stack to reverse a string — a classic demonstration of LIFO behavior:

1. **Push** every character of the input string onto the stack (first character goes in first, last character goes in last)
2. **Pop** all characters off the stack (last character comes out first, first character comes out last)
3. The characters naturally come out in **reverse order**

Think of it like putting cards face-down in a pile one by one, then flipping the pile over — the bottom card is now on top.

Example: `reverse("HELLO")` → push H, E, L, L, O → pop O, L, L, E, H → `"OLLEH"`

 Pseudocode:

```

global: character []: stack ← {100 undefined}
global: integer: sp ← 0

◦ string: reverse(string: inputStr)
  integer: n ← length of inputStr
  integer: i
  character: x, v
  string: outputStr ← ""
  for (increase i from 1 to n by 1)
    x ← the i-th character of string inputStr
    push(x)
  endfor
  while (sp ≠ 0)
    v ← pop()
    append v to outputStr
  endwhile
  return outputStr

◦ push (character: x)
  sp ← sp + 1
  stack[sp] ← x

◦ character: pop()
  character: retvar ← stack[sp]
  sp ← sp - 1
  return retvar

```

Category: Searching & Subarray Problems

2.20 — Maximum Subarray (Brute Force)

 **Year & Exam:** April 2025 (2025S)

 **Explanation:**

The **Maximum Subarray Problem** asks: "What contiguous slice of this array has the largest sum?" This is a famous problem in computer science (Kadane's algorithm solves it in $O(n)$, but this solution uses the $O(n^2)$ brute-force approach).

The algorithm tries **every possible subarray**:

- Outer loop: picks the starting index `i`
- Inner loop: extends the end index `j`, adding each new element to the running sum
- If the current sum exceeds the max, update the record

Think of it like trying every possible window position and size on a number line, keeping track of which window captures the most value.

Example: `{-2, 1, -3, 4, 1, 2, 1, 5, 4}` → best subarray is `{4, 1, 2, 1, 5, 4}` wait actually it's `{4, 1, 2, 1, -5, 4}`... The answer is indices 4–7 with sum 6 (subarray: `{4, -1, 2, 1}`)... Per the problem: sum=6, first=4, last=7.

 **Pseudocode:**

```

◦ maximumSubarray (integer []: T)
  integer: n ← the number of elements in T
  integer: i, j
  integer: first, last /* the first and last indices of the subarray */
  integer: sum
  integer: max ← T[1] - 1
  for (increase i from 1 to n by 1)
    sum ← 0
    for (increase j from i to n by 1)
      sum ← sum + T[j]
      if (sum > max)
        first ← i
        last ← j
        max ← sum
      endif
    endfor
  endfor
  output max, first, last

```

2.21 — Subarray Sum

 **Year & Exam:** October 2025 (2025A)

 **Explanation:**

Given an array of **positive integers** and a target sum, find a contiguous subarray that adds up to exactly that target. This is like looking for a consecutive run of numbers on a receipt that totals a specific amount.

The algorithm uses a **pruned brute-force** approach:

- Try every starting position
- Extend the subarray one element at a time, accumulating the sum
- If the sum **equals** the target → success, print the indices and return
- If the sum **exceeds** the target → stop extending (since all numbers are positive, adding more will only make it larger)

This early exit makes it more efficient than checking every possible subarray.

 **Pseudocode:**

```

◦ subArraySum(integer []: arr, integer: targetSum)
  integer: sum, start, end
  integer: N ← the number of elements in arr
  for (increase start from 1 to N by 1)
    sum ← 0
    for (increase end from start to N by 1)
      sum ← sum + arr[end]
      if (sum = targetSum)
        output start, end
        return
      elseif (sum > targetSum)
        exit the for block marked alpha
      endif
    endfor // alpha
  endfor
  output "No subarray found"

```

Category: OOP & Class Concepts

2.22 — Class Inheritance Output Trace

 **Year & Exam:** October 2025 (2025A)

 **Explanation:**

This is an **output tracing** problem testing your understanding of **class inheritance and method overriding**.

- `Rectangle` has width and height (2D)
- `Cuboid` extends `Rectangle` and adds depth (3D)
- The `enlarge` method adds corresponding dimensions of another shape to the calling object

When `f1.enlarge(f2)` is called, the `Rectangle f1(5, 2)` has its dimensions increased by the `Rectangle` portion of `Cuboid f2(9, 4, 7)`:

- Width: $5 + 9 = 14$
- Height: $2 + 4 = 6$
- (Depth is ignored because `f1` is a `Rectangle`, not a `Cuboid`)

Expected Output: `(14, 6)`

 Pseudocode:

```
Rectangle: f1 ← Rectangle(5, 2)
Cuboid: f2 ← Cuboid(9, 4, 7)
f1.enlarge(f2)
f1.output()

// Expected Output: (14, 6)
```

2.23 — Markov Chain (Bicycle Ridership)

 Year & Exam: October 2024 (2024A)

 Explanation:

A **Markov chain** models transitions between states where the next state depends only on the current state. Here, the states are "cycling" and "not cycling."

The transition probabilities form a **matrix**:

- 70% of cyclists continue cycling ($p_a = 1 - 0.3 = 0.7$)
- 30% of cyclists stop ($p_c = 0.3$)
- 2% of non-cyclists start cycling ($p_b = 0.02$)
- 98% of non-cyclists remain non-cyclists ($p_d = 1 - 0.02 = 0.98$)

The calculation is a simple **matrix-vector multiplication**: multiply the transition matrix by the current population vector to get next year's populations.

Result: cycling = $0.7 \times 5000 + 0.02 \times 100000 = 3500 + 2000 = 5500$, noncycling = $0.3 \times 5000 + 0.98 \times 100000 = 1500 + 98000 = 99500$

 Pseudocode:

```
real: pc ← 0.3
real: pb ← 0.02
real: pa ← (1 - pc)
real: pd ← (1 - pb)
integer []: N ← {5000, 100000}
real [,]: P ← {{pa, pb}, {pc, pd}}
real: cycling, noncycling
cycling ← P[1,1] * N[1] + P[1,2] * N[2]
noncycling ← P[2,1] * N[1] + P[2,2] * N[2]
output cycling, noncycling
```

Level 3 — Advanced

Very hard to grasp concepts with code implementation. These involve complex data structures (trees, linked lists, graphs, hash tables), advanced recursion, mathematical series approximations, and multi-step algorithms.

Category: Linked Lists

3.1 — Doubly Linked List — Delete Last Element

 **Year & Exam:** April 2025 (2025S)

 **Explanation:**

A **doubly linked list** is like a chain where each link knows both its **previous** and **next** neighbors. Removing the last element requires navigating to the end first.

The procedure handles three scenarios:

1. **Empty list** (`listHead` is undefined) → print "empty" (nothing to delete)
2. **Single element** (last element has no `prev`) → set `listHead` to undefined (list becomes empty)
3. **Multiple elements** → walk to the last element, then sever the connection by setting the second-to-last element's `next` to undefined

Think of it like unhooking the last train car: you walk to the end of the train, then disconnect the coupling.

 **Pseudocode:**

```
global: ListElement: listHead

◦ deleteLast()
  ListElement: current
  if (listHead is undefined) // empty list
    output "empty"
  else
    current ← listHead
    while (current.next is not undefined)
      current ← current.next
    endwhile
    if (current.prev is not undefined) // multiple elements are present
      current.prev.next ← undefined
    else // only one element is present
      listHead ← undefined
    endif
  endif
endif
```

3.2 — Reverse Singly Linked List (Recursive)

 **Year & Exam:** October 2025 (2025A)

 **Explanation:**

Reversing a linked list using **recursion** — one of the trickiest operations to visualize. The idea is like flipping a line of people:

1. **Recurse** to the end of the list to find the new head
2. On the way back (unwinding), **flip each connection** — each node now points to the node that used to point to *it*

The `elm` parameter carries the "previous" node so each node knows who to point back to. Initially, `elm` is undefined (the first node will become the last, pointing to nothing).

Trace for list `A → B → C` :

- Call `reverseList(A, undefined)` → `reverseList(B, A)` → `reverseList(C, B)`
- C is the last node → `listHead = C`
- Unwinding: C.next = B, B.next = A, A.next = undefined
- Result: `C → B → A`

 Pseudocode:

```

o ListElement: reverseList (ListElement: head, ListElement: elm)
  ListElement: listHead
  if (head.next is not undefined)
    listHead ← reverseList(head.next, head)
  else
    listHead ← head
  endif
  head.next ← elm
  return listHead

```

3.3 — Insert After in Singly Linked List

 Year & Exam: October 2025 (2025A)

 Explanation:

Inserting a new element *after* a specific existing element in a singly linked list. Think of it like cutting into a queue — you find the person you want to stand behind, then squeeze in.

The algorithm:

1. **Search** — walk through the list looking for the element with `targetData`
2. **Insert** — create a new element, point it to what the target currently points to, then point the target to the new element

The order of pointer reassignment matters! If you connect `x → y` before `y → x.next`, you'd lose the rest of the list.

Before: `... → [target] → [next] → ...` **After:** `... → [target] → [NEW] → [next] → ...`

 Pseudocode:

```

global: Element: head // stores first element in the list

o insertAfter (string: targetData, string: newData)
  Element: x, y
  x ← head
  while(x is not undefined)
    if (x.data = targetData)
      y ← Element(newData)
      y.next ← x.next
      x.next ← y
      exit the while block
    else
      x ← x.next
    endif
  endwhile

```

3.4 — Add Node at Position in Singly Linked List

 Year & Exam: October 2024 (2024A)

 Explanation:

This procedure inserts a new node at a **specific position** in the list. Two cases:

1. **Position 1 (insert at head):** The new node becomes the new head, pointing to the old head. Like cutting in front of everyone in line.
2. **Any other position:** Navigate to the node just *before* the target position (the `prev` node), then splice the new node in between `prev` and `prev.next`.

The loop runs `pos - 2` times (from 2 to `pos - 1`) to land on the correct predecessor node. If `pos = 2`, the loop doesn't execute at all — `prev` stays as `ListHead`.

 Pseudocode:

```

global: ListNode: listHead // stores the first node in the list

o addNode(integer: pos, character: val)
  ListNode: prev, newNode
  integer: i
  newNode ← ListNode()
  newNode.val ← val
  if (pos is equal to 1)
    newNode.next ← listHead
    listHead ← newNode
  else
    prev ← listHead
    /* if pos is equal to 2, the following iteration process is not executed */
    for (increase i from 2 to pos - 1 by 1)
      prev ← prev.next
    endfor
    newNode.next ← prev.next
    prev.next ← newNode
  endif

```

3.5 — Linear Circular Linked List Insertion

 **Year & Exam:** April 2024 (2024S)

 **Explanation:**

A **circular linked list** is like a ring — the last element points back to the first (head), creating a loop with no "end."

Inserting at the end of a circular list:

1. **Empty list:** The new node becomes the head and points to itself (a ring of one)
2. **Non-empty list:** Walk around the ring until you find the node whose `next` is `listHead` (that's the last node), then insert the new node between the last node and the head

The traversal `while (tmp.next is not listHead)` is the key — in a circular list, you can't check for `undefined`; instead, you look for the node that points back to the beginning.

 **Pseudocode:**

```

global: ListElement: listHead

o Insert(integer: newItem)
  ListElement: tmp, newNode
  newNode ← ListElement(newItem)
  if (listHead is undefined)
    listHead ← newNode
    listHead.next ← listHead
  else
    tmp ← listHead
    while (tmp.next is not listHead)
      tmp ← tmp.next
    endwhile
    tmp.next ← newNode
    newNode.next ← listHead
  endif

```

Category: Trees

3.6 — Identical Binary Trees Check

 **Year & Exam:** April 2025 (2025S)

 **Explanation:**

Two binary trees are **identical** if they have the same structure AND the same values at every corresponding position. This function uses **recursion** to check — think of it as comparing two family trees branch by branch:

1. Both nodes are undefined → They match (two empty branches are identical) →
2. Only one is undefined → Structure differs →
3. Values differ →
4. Values match → Recursively check *both* the left subtrees AND the right subtrees

This is a classic **tree traversal** pattern. The function only returns `true` if *every single node pair* passes all checks.

Pseudocode:

```

o boolean isSameTree(TreeNode: p, TreeNode: q)
  boolean: checkLeft, checkRight
  if (p = undefined and q = undefined)
    return true
  endif
  if (p = undefined or q = undefined)
    return false
  endif
  if (p.val ≠ q.val)
    return false
  endif
  checkLeft ← isSameTree(p.left, q.left)
  checkRight ← isSameTree(p.right, q.right)
  return checkLeft and checkRight

```

3.7 — Binary Tree Preorder Traversal (Using Stack)

Year & Exam: April 2024 (2024S)

Explanation:

Preorder traversal visits nodes in the order: **Root** → **Left** → **Right**. While this is naturally recursive, this implementation uses an **explicit stack** to simulate the recursion iteratively.

The clever trick: push the **right child first**, then the **left child**. Since a stack is LIFO, the left child will be popped (processed) first — exactly what preorder requires!

Step-by-step for a tree with root A, left B, right C:

1. Push A → Stack: [A]
2. Pop A, output "A", push C then B → Stack: [C, B]
3. Pop B, output "B" → Stack: [C]
4. Pop C, output "C" → Stack: []

Output: A, B, C (Root, Left, Right)

Pseudocode:

```

o preorder (Node: root)
  Node []: stack ← {undefined, ..., undefined} // an array with sufficient number of elements
  Node: v
  integer: sp ← 1 // The stack pointer
  stack[sp] ← root // Push root to the stack
  while (sp is not 0)
    v ← stack[sp] // Pop an element from the stack
    output v.info
    sp ← sp - 1
    if (v.right is not undefined)
      sp ← sp + 1
      stack[sp] ← v.right
    endif
    if (v.left is not undefined)
      sp ← sp + 1
      stack[sp] ← v.left
    endif
  endwhile

```

3.8 — Binary Search Tree Node Removal

 **Year & Exam:** October 2025 (2025A)

 **Explanation:**

Removing a node from a **Binary Search Tree (BST)** is one of the trickiest tree operations because you must maintain the BST property ($\text{left} < \text{root} < \text{right}$) after removal.

There are three cases:

1. **No children (leaf):** Simply remove it — like plucking a leaf from a tree
2. **One child:** Replace the node with its child — the child "moves up"
3. **Two children (the hard case):** Find the **in-order successor** (smallest value in the right subtree), copy its value to the current node, then recursively delete the successor

The in-order successor is found by going right once, then left as far as possible. This node is guaranteed to have at most one child (a right child), making its deletion simpler.

 **Pseudocode:**

```

o Node: remove (Node: node, integer: key)
  Node: node2
  if (node is undefined)
    return undefined
  elseif (key < node.key)
    node.left ← remove (node.left, key)
  elseif (key > node.key)
    node.right ← remove (node.right, key)
  elseif (node.right is undefined)
    return node.left
  else
    node2 ← node.right
    while (node2.left is not undefined)
      node2 ← node2.left
    endwhile
    node.key ← node2.key
    node.right ← remove (node.right, node2.key)
  endif
  return node

```

Category: Graph Algorithms

3.9 — Breadth-First Search (BFS)

 **Year & Exam:** October 2025 (2025A)

 **Explanation:**

BFS explores a graph level by level — like ripples spreading outward when you drop a stone in water. It visits all neighbors of a vertex before moving to the next level.

BFS uses a **Queue** (FIFO — First In, First Out):

1. Start at vertex **k**, mark it as visited, and add it to the queue
2. Dequeue a vertex, output it, then enqueue all its **unvisited** neighbors
3. Repeat until the queue is empty

The **graph** is represented as an **adjacency matrix**: `graph[i,j] = 1` means vertices *i* and *j* are connected.

Trace for `traverse(1)` on the given graph:

1. Queue: [1] → dequeue 1, output 1, enqueue neighbors 2, 4
2. Queue: [2, 4] → dequeue 2, output 2, enqueue neighbors 3, 5
3. Queue: [4, 3, 5] → dequeue 4, output 4, enqueue neighbor 5 (already visited, skip)
4. Queue: [3, 5] → dequeue 3, output 3 → dequeue 5, output 5

Output: 1, 2, 4, 3, 5

 Pseudocode:

```

global: integer [,]: graph = {{0, 1, 0, 1, 0},
                               {1, 0, 1, 0, 1},
                               {0, 1, 0, 0, 0},
                               {1, 0, 0, 0, 1},
                               {0, 1, 0, 1, 0}}

◦ traverse(integer: k)
  Queue: queue ← Queue()
  boolean []: visited ← {false, false, false, false, false}
  integer: v, i
  queue.enqueue(k)
  visited[k] ← true
  while (not queue.isEmpty())
    v ← queue.dequeue()
    output v
    for (increase i from 1 to 5 by 1)
      if (graph[v, i] = 1 and visited[i] = false)
        queue.enqueue(i)
        visited[i] ← true
      endif
    endfor
  endwhile

// Expected Output for traverse(1): 1, 2, 4, 3, 5

```

3.10 — Depth-First Search (DFS)

 Year & Exam: October 2024 (2024A) Explanation:

DFS explores a graph by going as deep as possible along each branch before backtracking — like exploring a maze by always turning left until you hit a dead end, then backing up.

Unlike BFS (which uses a queue), DFS uses **recursion** (which implicitly uses the call stack):

1. Mark the current vertex as visited and output it
2. For each unvisited neighbor, recursively explore from that neighbor
3. When all neighbors are explored, backtrack

Trace for `traverse(1)` on the given graph:

1. Visit 1 → neighbor 2 is unvisited → recurse
2. Visit 2 → neighbor 3 is unvisited → recurse
3. Visit 3 → no unvisited neighbors → backtrack to 2
4. Back at 2 → neighbor 5 is unvisited → recurse
5. Visit 5 → neighbor 4 is unvisited → recurse
6. Visit 4 → no unvisited neighbors → backtrack

Output: `1, 2, 3, 5, 4`

 Pseudocode:

```

global: integer: n ← 5
global: integer [,]: graph ← {{0, 1, 0, 1, 0},
                             {1, 0, 1, 0, 1},
                             {0, 1, 0, 0, 0},
                             {1, 0, 0, 0, 1},
                             {0, 1, 0, 1, 0}}
global: boolean []: visited ← {false, false, false, false, false}

◦ traverse (integer: k)
  integer: i
  visited[k] ← true
  output k
  for (increase i from 1 to n by 1)
    if (graph[k][i] = 1 and visited[i] = false)
      traverse(i)
    endif
  endfor

// Expected Output for traverse(1): 1, 2, 3, 5, 4

```

Category: Advanced Data Structures

3.11 — Balanced Brackets Check

 **Year & Exam:** April 2024 (2024S)

 **Explanation:**

This is a classic **stack problem** — checking if brackets in an expression are properly matched and nested, like verifying that every open door has a corresponding close.

Rules:

- (matches), { matches }, [matches]
- Brackets must be **properly nested** — ([]) is invalid, ([[]) is valid

The algorithm:

1. See an **opening bracket** → push it onto the stack (remember it for later)
2. See a **closing bracket** → pop from the stack and check if it matches
3. If the stack is empty when you need to pop → unmatched closing bracket → ✗
4. After processing all characters, stack should be empty → all brackets matched → ✓

Example: {([O])} → push {, push [, push (, pop (matches) ✓, pop [matches] ✓, pop { matches } ✓, stack empty ✓ → **true**

 **Pseudocode:**

```

global: character [][]: brackets ← {
  {"(", ")"},
  {"{", "}"},
  {"[", "]" }
}

◦ boolean: are_brackets_balanced(character []: expr)
Stack: stack ← Stack()
character: c, stacked_bracket
for (c in expr)
  if (is_opening_bracket(c))
    stack.push(c)
  else
    if (stack.isEmpty())
      return false
    endif
    stacked_bracket ← stack.pop()
    if (get_closing_bracket(stacked_bracket) ≠ c)
      return false
    endif
  endif
endfor
return stack.isEmpty()

◦ boolean: is_opening_bracket(character: c)
character []: chars
for (chars in brackets)
  if (chars[1] = c)
    return true
  endif
endfor
return false

◦ character: get_closing_bracket(character: c)
character []: chars
for (chars in brackets)
  if (chars[1] = c)
    return chars[2]
  endif
endfor
return undefined

```

3.12 — Priority Queue Trace

 **Year & Exam:** October 2024 (2024A)

 **Explanation:**

A **priority queue** is like the waiting room at an emergency room — patients aren't seen in the order they arrive, but by the severity of their condition (priority). Lower number = higher priority.

In this trace:

1. Enqueue E(pri=3), F(pri=2), G(pri=1), H(pri=1) → Queue: [G, H, F, E] (sorted by priority)
2. Dequeue → G (highest priority) is removed
3. Dequeue → H (next highest) is removed
4. Enqueue I(pri=1), J(pri=1) → Queue: [I, J, F, E]
5. Dequeue → I is removed
6. Enqueue K(pri=2), L(pri=3), M(pri=1)
7. Dequeue remaining: J(1), M(1), F(2), K(2), E(3), L(3)

Expected Output: "I", "J", "M", "F", "K", "E", "L" ... wait, let me re-trace. After step 5, queue has [J, F, E]. Then add K(2), L(3), M(1). Queue: [J, M, F, K, E, L]. Dequeue all:

Expected Output: "J", "M", "F", "K", "E", "L"

 **Pseudocode:**

```

o prioSched()
  PrioQueue: prioQueue ← PrioQueue()
  prioQueue.enqueue("E", 3)
  prioQueue.enqueue("F", 2)
  prioQueue.enqueue("G", 1)
  prioQueue.enqueue("H", 1)
  prioQueue.dequeue() /* The return value is ignored */
  prioQueue.dequeue() /* The return value is ignored */
  prioQueue.enqueue("I", 1)
  prioQueue.enqueue("J", 1)
  prioQueue.dequeue() /* The return value is ignored */
  prioQueue.enqueue("K", 2)
  prioQueue.enqueue("L", 3)
  prioQueue.enqueue("M", 1)
  while (prioQueue.size() is not equal to 0)
    output prioQueue.dequeue()
  endwhile

// Expected Output: "J", "M", "F", "K", "E", "L"

```

3.13 — Hash Table Insertion (Linear Probing)

 **Year & Exam:** October 2024 (2024A)

 **Explanation:**

A **hash table** is like a giant locker room where each item's locker number is calculated from its key. The **hash function** converts a key into an index:

$(\text{key} \bmod 1000) + 1$.

But what happens when two keys map to the same locker? That's a **collision**. This implementation uses **linear probing** to resolve collisions:

- If the target locker is occupied, check the next one
- If you reach the end (locker 1000), wrap around to locker 1
- Keep going until you find an empty locker

Think of it like parking at a crowded mall — if your preferred spot is taken, you drive forward one spot at a time until you find an empty one.

 **Pseudocode:**

```

global: integer [][]: hashTable ← {Elements comprising 1000 {undefined}}
global: integer: size ← 1000

o integer: hashFunction (integer: key)
  return (key mod size) + 1

o insertData(integer: key, integer: data)
  integer: index
  index ← hashFunction (key)
  while (hashTable[index][1] ≠ undefined)
    if (index = size)
      index ← 1
    else
      index ← index + 1
    endif
  endwhile
  hashTable[index] ← {key, data}

```

Category: Mathematical Series & Approximations

3.14 — Cosine Maclaurin Series Approximation

 **Year & Exam:** April 2025 (2025S)

 **Explanation:**

The **Maclaurin series** is a way to approximate mathematical functions using an infinite sum of polynomial terms. For cosine:

$$\cos(y) = 1 - \frac{y^2}{2!} + \frac{y^4}{4!} - \frac{y^6}{6!} + \dots$$

Instead of computing each term from scratch (which involves large factorials), this code uses a **recurrence relation** — each term is computed from the previous one:

$$\text{term}_n = \text{term}_{n-1} \times \frac{-y^2}{2n(2n-1)}$$

This is like a factory assembly line — each worker only needs to make a small modification to the previous worker's output, rather than starting from raw materials.

The loop continues until the term becomes negligibly small (< 0.000000001), ensuring precision.

Note: Input is in degrees, so it's first converted to radians using $y = z \times \pi/180$.

 **Pseudocode:**

```

◦ real: cosine (real: z)
  real: y ← z * pi / 180 // convert from degrees to radians
  real: term ← 1
  real: cosy ← term
  integer: n ← 0
  while (absolute value of term > 0.000000001)
    n ← n + 1
    term ← term * (-1 * (y squared / (2 * n * (2 * n - 1))))
    cosy ← cosy + term
  endwhile
  return cosy

```

3.15 — Maclaurin Expansion of Sin(x)

 **Year & Exam:** April 2024 (2024S)

 **Explanation:**

Similar to the cosine approximation, this computes `sin(x)` using its Maclaurin series:

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

Again, a **recurrence relation** is used to derive each term from the previous:

$$v_n = -v_{n-1} \times \frac{x^2}{(k-1) \times k}$$

Where `k` increases by 2 each iteration (1 → 3 → 5 → 7...), corresponding to the odd-only exponents in the sine series.

The negative sign alternates the terms between addition and subtraction, and the denominator grows quickly (as factorial terms do), causing the terms to shrink rapidly toward zero.

 **Pseudocode:**

```

◦ real: m_sin(real: x)
  real: vn ← x
  real: k ← 1
  real: sum ← vn
  real: epsi ← 1 * 10^-7
  while (abs(vn) > epsi) // abs(vn) returns the absolute value of vn
    k ← k + 2
    vn ← -vn * x^2 / ((k - 1) * k)
    sum ← sum + vn
  endwhile
  return sum

```

Category: Simulation & Probability

3.16 — Craps Monte Carlo Simulation

 **Year & Exam:** April 2025 (2025S)

💡 Explanation:

Monte Carlo simulation approximates probabilities by running thousands of random trials — like playing a casino game 10,000 times to estimate your chances of winning.

Craps rules simplified:

1. Roll two dice, add them up
2. Sum = 7 or 11 → **immediate win**
3. Sum = 2, 3, or 12 → **immediate loss**
4. Any other sum (4, 5, 6, 8, 9, 10) → this becomes your "point." Keep rolling until you either:
 - Roll your point again → **win**
 - Roll a 7 → **lose**

The theoretical win probability is exactly $244/495 \approx 0.4929\dots$

The program runs 10,000 games, counts wins, and calculates:

- **Result** = wins / total games (experimental probability)
- **Relative error** = how far off from the theoretical value

📝 Pseudocode:

```
integer: wins_sum ← 0
integer: lose_sum ← 0
integer: n ← 10000
integer: i, dice1, dice2, sum, newsum
real: result, pt ← (244 / 495)
for (increase i from 1 to n by 1)
  dice1 ← random_int(1, 6)
  dice2 ← random_int(1, 6)
  sum ← dice1 + dice2
  if (sum = 7 or sum = 11)
    wins_sum ← wins_sum + 1
  elseif (sum = 2 or sum = 3 or sum = 12)
    lose_sum ← lose_sum + 1
  else
    do
      dice1 ← random_int(1, 6)
      dice2 ← random_int(1, 6)
      newsum ← dice1 + dice2
      if (newsum = sum)
        wins_sum ← wins_sum + 1
      elseif (newsum = 7)
        lose_sum ← lose_sum + 1
      endif
    while (newsum ≠ sum and newsum ≠ 7)
  endif
endfor
result ← wins_sum / n
output result, absolute value of ((result - pt) / pt)
```

Category: Text & Data Analysis Algorithms**3.17 — Magic Square Check**

 **Year & Exam:** October 2025 (2025A)

💡 Explanation:

A **3×3 magic square** is a grid containing numbers 1–9 where every row, column, and diagonal sums to 15. Think of it as the ultimate balanced grid — no matter which line you draw through it, the total is the same.

The algorithm pre-computes all sums in a single pass:

- **Row sums** (`rowSum[i]`) — sum of each row
- **Column sums** (`colSum[i]`) — sum of each column
- **Diagonal sums** (`dia1Sum` , `dia2Sum`) — main diagonal (↘) and anti-diagonal (↗)

Then it verifies:

1. Both diagonals must be equal
2. Every row sum must equal every column sum, and both must equal the diagonal sum

Example of a valid magic square:

```
2 7 6
9 5 1
4 3 8
```

 Pseudocode:

```
o boolean: checkMagicSquare (integer [,]: m)
  integer: i, j, k
  integer: dia1Sum ← 0
  integer: dia2Sum ← 0
  integer []: rowSum ← {0, 0, 0}
  integer []: colSum ← {0, 0, 0}
  for (increase i from 1 to 3 by 1)
    for (increase j from 1 to 3 by 1)
      rowSum[i] ← rowSum[i] + m[i, j]
      colSum[i] ← colSum[i] + m[j, i]
    endfor
    dia1Sum ← dia1Sum + m[i, i]
    dia2Sum ← dia2Sum + m[i, 3 - i + 1]
  endfor
  if (dia1Sum ≠ dia2Sum)
    return false
  endif
  for (increase k from 1 to 3 by 1)
    if ((rowSum[k] ≠ colSum[k]) or (rowSum[k] ≠ dia1Sum))
      return false
    endif
  endfor
  return true
```

3.18 — N-Grams Generation

 Year & Exam: April 2024 (2024S)

 Explanation:

N-grams are sliding windows of N consecutive words from a text. They're fundamental in natural language processing (NLP) for tasks like predictive text, spam detection, and language modeling.

- **Unigram (n=1):** Each word individually → "I", "love", "coding"
- **Bigram (n=2):** Pairs of consecutive words → "I love", "love coding"
- **Trigram (n=3):** Triples → "I love coding"

The algorithm splits the text into words, then uses a sliding window of size `n`:

- The outer loop determines the starting position (up to `length - n + 1`)
- The inner loop concatenates `n` consecutive words into a single n-gram string

Example: `NGRAMS(2, "I love to code")` → outputs: `"I love "`, `"love to "`, `"to code "`

 Pseudocode:

```

◦ NGRAMS (integer: n, string: text)
  string []: words ← split(text)
  string: s
  integer: i, j, length
  length ← the number of elements in words
  for (increase i from 1 to length - n + 1 by 1)
    s ← ""
    for (increase j from i to i + n - 1 by 1)
      s ← s + words[j] + " "
    endfor
    output s
  endfor

```

3.19 — Inverse Document Frequency (IDF)

 Year & Exam: October 2025 (2025A)

 Explanation:

IDF measures how rare and important a word is across a collection of documents. Words that appear in many documents (like "the", "is") get low IDF scores, while rare words (like "pseudocode") get high scores.

$$IDF(t) = \log \left(\frac{N}{1 + df(t)} \right)$$

Where:

- N = total number of documents
- $df(t)$ = number of documents containing the term
- The $+1$ prevents division by zero

The algorithm scans each document, splits it into words, and checks if the term exists. The `exit` after finding the term ensures each document is counted at most once (even if the term appears multiple times in it).

Think of it like checking how many restaurants in a city serve sushi. If only 2 out of 100 serve it, sushi is "rare" → high IDF.

 Pseudocode:

```

◦ real: calcIDF (string: term, string []: corpus)
  integer: i, j, numDocs, numWords, termCount
  boolean: isContainsTerm
  real: idf
  string []: words
  termCount ← 0
  numDocs ← the number of elements in corpus
  for (increase i from 1 to numDocs by 1)
    isContainsTerm ← false
    words ← split(corpus[i])
    numWords ← the number of elements in words
    for (increase j from 1 to numWords by 1)
      if (words[j] = term)
        isContainsTerm ← true
        exit the for block marked alpha
      endif
    endfor // alpha
    if (isContainsTerm = true)
      termCount ← termCount + 1
    endif
  endfor
  idf ← log(numDocs / (1 + termCount)) /* Division is performed in data type real */
  return idf

```

3.20 — Jaccard Similarity Index

 Year & Exam: October 2025 (2025A)

 Explanation:

The Jaccard similarity measures how similar two sets are — it's the ratio of what they **share** to what they **have in total**:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

- **Intersection** ($A \cap B$): elements in BOTH sets
- **Union** ($A \cup B$): all unique elements from BOTH sets

Think of it like comparing two people's music playlists: if they share 5 songs and have 20 unique songs total between them, the Jaccard similarity is $5/20 = 0.25$.

The algorithm counts:

- **iCount**: number of common elements (intersection size)
- **uCount**: starts with **nB** and adds elements from A that aren't in B (union size)

Example: $A = \{1,2,3\}$, $B = \{2,3,4\} \rightarrow$ intersection = $\{2,3\}$ (iCount=2), union = $\{1,2,3,4\}$ (uCount=4) $\rightarrow J = 2/4 = 0.5$

 **Pseudocode:**

```

◦ real: jaccardSimilarity(integer []: A, integer []: B)
integer: nA ← number of elements in A
integer: nB ← number of elements in B
integer: iCount ← 0
integer: uCount ← nB
boolean: found
for (increase i from 0 to nA - 1 by 1)
    found ← false
    for (increase j from 0 to nB - 1 by 1) // alpha
        if (A[i] = B[j])
            iCount ← iCount + 1
            found ← true
            exit the for block marked alpha
        endif
    endfor
    if (found = false)
        uCount ← uCount + 1
    endif
endfor
return iCount / uCount /* Division is done in data type real */

```

3.21 — Cosine Similarity Calculation

 **Year & Exam:** April 2024 (2024S)

 **Explanation:**

Cosine similarity measures the angle between two vectors — if they point in the same direction, similarity is 1; if perpendicular, it's 0.

$$\text{similarity} = \frac{\vec{v}_1 \cdot \vec{v}_2}{|\vec{v}_1| \times |\vec{v}_2|} = \frac{\sum x_i \cdot y_i}{\sqrt{\sum x_i^2} \times \sqrt{\sum y_i^2}}$$

Unlike Jaccard (which compares sets), cosine similarity works with **magnitudes and directions**. It's widely used in:

- **Document similarity** — comparing documents by word frequency vectors
- **Recommendation systems** — "users who liked this also liked..."
- **Search engines** — matching queries to documents

The code computes three running sums in one pass: **sxy** (dot product), **sxx** (magnitude of v1 squared), **syy** (magnitude of v2 squared).

 **Pseudocode:**

```
// Assume that arrays v1 and v2 have the same number of one or more elements
// and that the arrays are not all-zero.
◦ real: calcSim (integer []: v1, integer []: v2)
  integer: i, x, y
  integer: sxx ← 0
  integer: syy ← 0
  integer: sxy ← 0
  for (increase i from 1 to the number of elements in v1 by 1)
    x ← v1[i]
    y ← v2[i]
    sxx ← sxx + x * x
    syy ← syy + y * y
    sxy ← sxy + x * y
  endfor
  return sxy / (square root of (sxx * syy))
```

3.22 — Extended Hamming Distance

 **Year & Exam:** October 2024 (2024A)

 **Explanation:**

This is an **extension** of the classic Hamming distance (see 2.9) that also handles strings of **different lengths**. The standard Hamming distance only works for equal-length strings, but this version also counts the extra characters in the longer string as differences.

The algorithm works in two phases:

1. **Compare shared positions:** Walk through both strings up to the length of the shorter one, counting mismatches
2. **Count leftover characters:** Any extra characters in the longer string are automatically counted as differences

Think of it like comparing two words: "cat" vs "castle" → compare c=c, a=a, t=s (1 mismatch) + 3 extra characters in "castle" = 4.

Example: `hammingDistance("101010", "111000111")` → 2 mismatches in first 6 chars + 3 extra chars = **5**

 **Pseudocode:**

```
◦ integer: hammingDistance (string: s1, string: s2)
  integer: distance, length1, length2, minLength, remainingLength
  length1 ← length of s1
  length2 ← length of s2
  distance ← 0
  minLength ← length1
  if (length2 < length1)
    minLength ← length2
  endif
  for (increase i from 1 to minLength by 1)
    if (the i-th character of string s1 is not equal to the i-th character of string s2)
      distance ← distance + 1
    endif
  endfor
  if (length1 > length2)
    remainingLength = length1 - length2
  else
    remainingLength = length2 - length1
  endif
  distance ← distance + remainingLength
  return distance
```

Category: Dynamic Programming & Recursion

3.23 — Longest Common Subsequence (LCS)

 **Year & Exam:** October 2024 (2024A)

 **Explanation:**

The **LCS problem** finds the longest sequence of characters that appears in both strings (not necessarily contiguous). It's a cornerstone of **dynamic programming** and is used in diff tools, DNA sequence alignment, and version control.

The recursive logic:

1. **Base case:** If either string is empty ($m=0$ or $n=0$) → LCS length is 0
2. **Characters match:** The last characters of both strings are the same → this character is part of the LCS, add 1 and check the remaining strings
3. **Characters don't match:** Try two possibilities — skip the last character of `str1`, or skip the last character of `str2` — and take the maximum

Think of it like two people reading two books and trying to find the longest sequence of words they both encountered in order.

Example: `lcs("ABXDZ", "ABCD", 5, 4)` → LCS is "ABD", length = `3`

 **Note:** This recursive approach has exponential time complexity. The dynamic programming (table-based) version is much more efficient, but this recursive version is elegant and tests understanding of recursion.

 **Pseudocode:**

```
integer: max(integer: a, integer: b)
  if (a > b)
    return a
  else
    return b
  endif

integer: lcs (string: str1, string: str2, integer: m, integer: n)
  if (m = 0 or n = 0)
    return 0
  endif
  if (the m-th character of string str1 = the n-th character of string str2)
    return 1 + lcs(str1, str2, m - 1, n - 1)
  else
    return max(lcs(str1, str2, m, n - 1), lcs(str1, str2, m - 1, n))
  endif
```

Category: Advanced Math & Matrix Operations

3.24 — Markov Chain (Weather Prediction)

 **Year & Exam:** October 2024 (2024A)

 **Explanation:**

This extends the Markov chain concept from 2.23 to a **3-state system** (rainy, sunny, cloudy) and computes the **3-day transition matrix** using matrix multiplication.

The transition matrix `pmat` encodes probabilities:

	Rain	Sun	Cloud
Rain	[0.3 0.4 0.3]		
Sun	[0.2 0.7 0.1]		
Cloud	[0.25 0.5 0.25]		

To find probabilities after 3 days, we need P^3 (the transition matrix raised to the 3rd power). The code computes this by multiplying the matrix by itself twice:

- First iteration: `dmat = pmat × pmat = P2`
- Second iteration: `dmat = P2 × pmat = P3`

The triple nested loop implements standard matrix multiplication: each element `smat[i,j]` is the sum of `dmat[i,k] * pmat[k,j]` for all k .

 **Pseudocode:**

```

real [,]: pmat ← {{0.3, 0.4, 0.3}, {0.2, 0.7, 0.1}, {0.25, 0.5, 0.25}}
real [,]: dmat ← pmat
real [,]: smat
integer: i, j, k, m
for (increase m from 1 to 2 by 1)
  smat ← {{0, 0, 0}, {0, 0, 0}, {0, 0, 0}}
  for (increase i from 1 to 3 by 1)
    for (increase j from 1 to 3 by 1)
      for (increase k from 1 to 3 by 1)
        smat[i, j] ← smat[i, j] + dmat[i, k] * pmat[k, j]
      endfor
    endfor
  endfor
  dmat ← smat
endfor
output dmat

```

3.25 — Array Summarization (Five-Number Summary / Quantiles)

 **Year & Exam:** October 2024 (2024A)

 **Explanation:**

The **five-number summary** is a staple of statistics: **minimum**, **Q1 (25th percentile)**, **median (50th percentile)**, **Q3 (75th percentile)**, and **maximum**. It gives you a complete picture of data distribution and is the basis for box plots.

The `findRank` function extracts a value at a specific quantile position:

- `q = 0` → minimum (first element)
- `q = 0.25` → lower quartile
- `q = 0.5` → median
- `q = 0.75` → upper quartile
- `q = 1` → maximum (last element)

The formula `floor(q * (n-1))` converts the quantile to an index. For a 10-element array (indices 0–9): $q=0.5 \rightarrow \text{floor}(0.5 \times 9) = \text{floor}(4.5) = 4 \rightarrow$ 5th element (index 4+1 in 1-based indexing).

Example: For `{0.1, 0.2, ..., 1.0}`: result = `{0.1, 0.3, 0.5, 0.7, 1.0}`

 **Pseudocode:**

```

◦ real: findRank (real []: sortedData, real: q)
  integer: j
  j ← floor(q * (the number of elements in sortedData - 1))
  // floor returns the closest integer less than or equal to a given
  // number, e.g. floor (7.75) returns 7.
  return sortedData[j + 1]

◦ real []: summarize (real []: sortedData)
  real []: rankData ← {} /* array with 0 elements */
  real []: q ← {0, 0.25, 0.5, 0.75, 1}
  integer: i
  for (increase i from 1 to the number of elements in q by 1)
    add the return value of findRank (sortedData, q[i]) to the end of rankData
  endfor
  return rankData

// Expected Output for summarize({0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1}): {0.1, 0.3, 0.5, 0.7, 1}

```



Quick Reference Index

#	TOPIC	LEVEL	EXAM	CATEGORY
1.1	Letter Grade Evaluation	1	2024S	Conditional Logic
1.2	Bus Ticket Price	1	2024S	Conditional Logic
1.3	Leap Year Verification	1	2025S	Conditional Logic
1.4	Coupon Calculation	1	2024A	Conditional Logic
1.5	$(\sqrt{x} + \sqrt{y})^2$ Calculation	1	2025S	Simple Math
1.6	Sum of Even Numbers	1	2024S	Simple Math
1.7	Sum of Digits	1	2024S	Simple Math
1.8	Seconds to HH:MM:SS	1	2024S	Simple Math
1.9	Sum of Even Elements	1	2025A	Simple Math
1.10	Reverse Digits	1	2025A	Simple Math
1.11	Perfect Number	1	2025S	Simple Math
1.12	Geometric Mean	1	2024A	Simple Math
1.13	Quadratic Equation Roots	1	2024A	Simple Math
2.1	Binary to Decimal	2	2025S	Number Theory
2.2	Print Primes	2	2025A	Number Theory
2.3	Prime Factors	2	2024A	Number Theory
2.4	Count Set Bits	2	2024S	Bit Manipulation
2.5	Gray Code to Binary	2	2025S	Bit Manipulation
2.6	Fibonacci Sequence	2	2024S	Recursion
2.7	Palindrome Check (String)	2	2024S	String/Array Ops
2.8	Palindrome Check (Array)	2	2024A	String/Array Ops
2.9	Hamming Distance	2	2025S	String/Array Ops
2.10	Contains Substring	2	2025A	String/Array Ops
2.11	Find Mode	2	2025A	String/Array Ops
2.12	Second-Largest Element	2	2024A	String/Array Ops
2.13	Selection Sort	2	2024S	Sorting
2.14	Insertion Sort	2	2024A	Sorting
2.15	Optimized Bubble Sort	2	2025A	Sorting
2.16	Counting Sort	2	2025S	Sorting
2.17	Stack Push/Pop	2	2025A	Stacks
2.18	Stack Operations	2	2024S	Stacks
2.19	String Reverse (Stack)	2	2025S	Stacks

#	TOPIC	LEVEL	EXAM	CATEGORY
2.20	Maximum Subarray	2	2025S	Searching
2.21	Subarray Sum	2	2025A	Searching
2.22	Class Inheritance Trace	2	2025A	OOP
2.23	Markov Chain (Bicycle)	2	2024A	Math/Matrix
3.1	Doubly Linked List Delete	3	2025S	Linked Lists
3.2	Reverse Linked List	3	2025A	Linked Lists
3.3	Insert After (Linked List)	3	2025A	Linked Lists
3.4	Add Node at Position	3	2024A	Linked Lists
3.5	Circular Linked List Insert	3	2024S	Linked Lists
3.6	Identical Binary Trees	3	2025S	Trees
3.7	Preorder Traversal (Stack)	3	2024S	Trees
3.8	BST Node Removal	3	2025A	Trees
3.9	BFS (Graph Traversal)	3	2025A	Graphs
3.10	DFS (Graph Traversal)	3	2024A	Graphs
3.11	Balanced Brackets	3	2024S	Advanced DS
3.12	Priority Queue Trace	3	2024A	Advanced DS
3.13	Hash Table (Linear Probing)	3	2024A	Advanced DS
3.14	Cosine (Maclaurin Series)	3	2025S	Math Series
3.15	Sin(x) (Maclaurin Series)	3	2024S	Math Series
3.16	Craps Monte Carlo	3	2025S	Simulation
3.17	Magic Square Check	3	2025A	Data Analysis
3.18	N-Grams Generation	3	2024S	Data Analysis
3.19	IDF Calculation	3	2025A	Data Analysis
3.20	Jaccard Similarity	3	2025A	Data Analysis
3.21	Cosine Similarity	3	2024S	Data Analysis
3.22	Extended Hamming Distance	3	2024A	Data Analysis
3.23	Longest Common Subsequence	3	2024A	Dynamic Programming
3.24	Markov Chain (Weather)	3	2024A	Matrix Operations
3.25	Quantiles / Five-Number Summary	3	2024A	Statistics